Persistent

Events, Property Binding and Data Binding

Persistent Interactive | Persistent University

Key Learning Points

- Event Binding
- Property Binding
- Two-way data binding

Event Binding

- User actions on the page result in DOM events.
- Handlers are written in the **component's class.**
- One way is with parentheses

```
<button (click)="save()">Save</button>
```

```
<input type="text" (keyup)="validate()"/>
```

```
<button on-click="save()">Save</button>
```

$event Object

- The inbuilt-object **'$event'** contains more information about the event that occurred such as mouse coordinates, which key code, target element etc.

```
<input (keyup)="onKey($event)">
```

- Retrieve the values in the handler.

```
onKey(event: any) {
        this.value = event.target.value ;
 }
```

Template Reference Variables

- Access elements' properties by using reference variable : #box.

```
<input #box (keyup)="onKey(box.value)">
```

- Define a variable name preceded by #.

```
<input #box (keyup)="0">
  <p>{{box.value}}</p>
```

Key.enter

- To capture the event when user types the ‘enter’ key.
- Angular's **keyup.enter** pseudo-event.

```
<input #box (keyup.enter)="onEnter(box.value)">
```

Clear the textbox

- Call multiple Javascript statements for one event.
- Using the template ref variable, we can reset the textbox.

```
<input #user
    (keyup.enter)="adduser(user.value)"
    (blur)="addUser(user.value); user.value='' ">
```

Property Binding

- **Property binding is only used to set the values. Cannot read back from it.**
- Set property of ‘src’
- Use of square brackets []

```
<img [src]="path" />
```

- ‘path’ is defined in the component’s class.

```
path:string="images/motog4.jpg";
```

Custom properties

- Here, ‘username’ is a custom property for the WelcomeComponent**.**
- **Sets the property ‘username’ with the value of variable ‘name’**

```
<welcome [username]="name" ></welcome>
```

- WelcomeComponent defined as below.

```
export class WelcomeComponent {
    name:string="Jane";
}
```

Interpolation

- **Interpolation** is One way data binding
- From Model to View

```
<h2>{{message}}—{{name}}</h2>
```

- But how to update changes in view back to model?

Interpolation vs Property binding

- Angular evaluates all expressions in double curly braces, **converts the expression results to strings**, and concatenates them with neighboring literal strings.
- Property binding does not convert the expression result to a string.
- Hence use property binding when there is a need to bind to variable types other than strings.
- Use Interpolation since src value evaluates to string.

```
<img src= {{ imageUrl }} >
```

- Use Property binding since ‘disabled’ must evaluate to Boolean.

```
<button [disabled]=“isAdmin”>
```

Two way Data Binding

```
@Component({
    selector:'welcome',
    template:`<input type="text" [(ngModel)]="name" /><h2>{{message}}-{{name}}</h2>`
})
```

- Banana in a box [()]
- The Model and View are always in Sync
- We must import the **FormsModule** and add it to the Angular module's imports list.

LifeCycle Hooks

- We can tap into the different phases with the help of lifecycle hooks.
- **ngOnChanges** - called after Angular sets a data-bound input property.
- **ngOnInit** - Angular initializes the data-bound input properties.
- **ngDoCheck** - after every run of change detection
- **ngAfterContentInit** - after component content initialized
- **ngAfterContentChecked** - after every check of component content
- **ngAfterViewInit** - After Angular creates the component’s view(s).
- **ngAfterViewChecked** - after every check of a component's view(s)
- **ngOnDestroy** - just before the component is destroyed

Implementing the lifecycle hooks

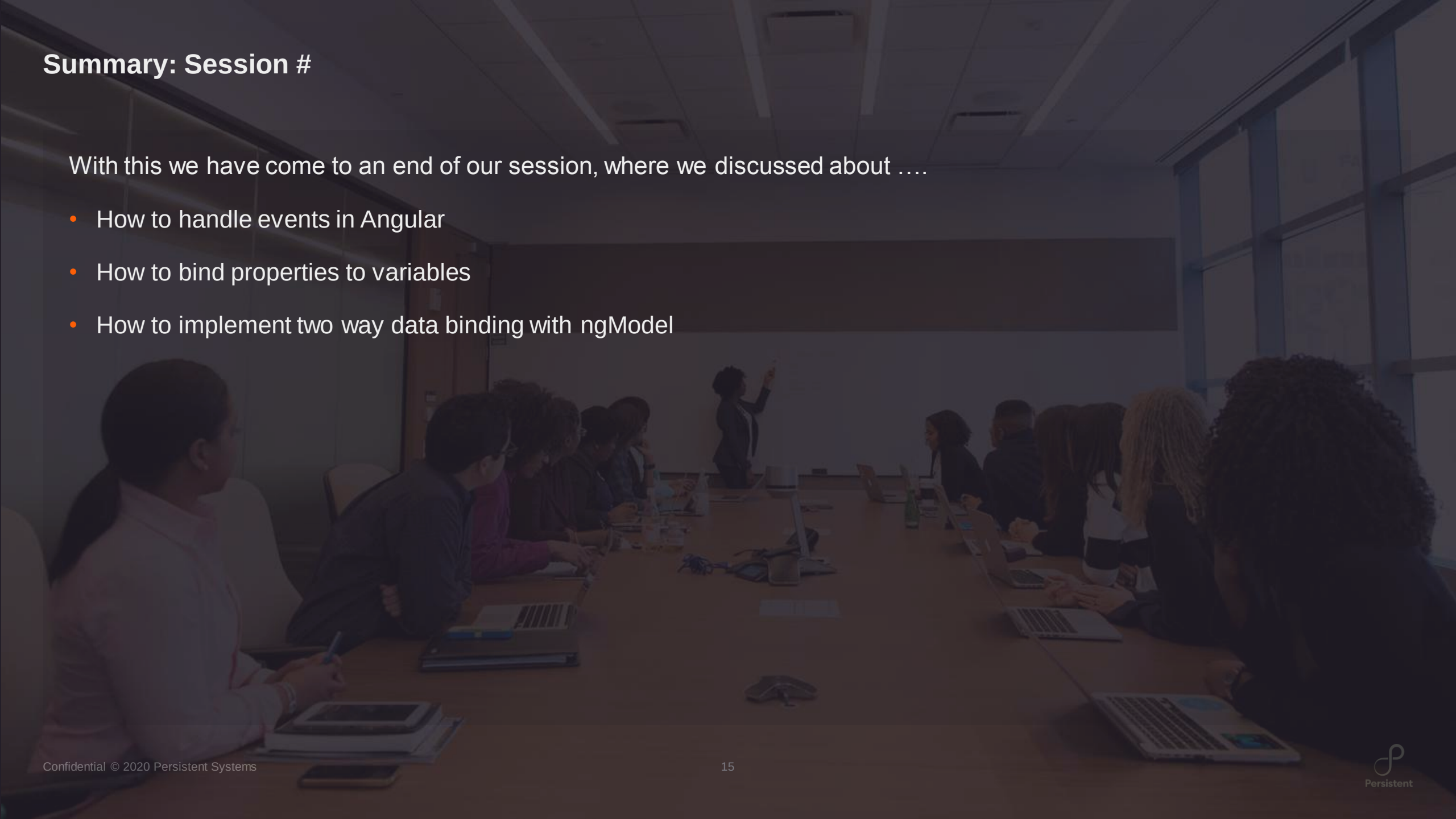
- Eg: Reference a DOM element in ngAfterViewInit()

```
ngAfterViewInit() {
        var canvas: any = this.cropcanvas.nativeElement;
}
```

Summary: Session

With this we have come to an end of our session, where we discussed about ….

- How to handle events in Angular
- How to bind properties to variables
- How to implement two way data binding with ngModel



Appendix

References

Thank You

Reference Links

- http://blog.angular-university.io/introduction-to-angular-2-fundamentals-of-components-events-properties-and-actions/
- Event binding
  - https://angular.io/docs/ts/latest/guide/user-input.html
  - https://angular.io/docs/ts/latest/guide/template-syntax.html#!#event-binding
- Property binding
  - https://angular.io/docs/ts/latest/guide/template-syntax.html#!#property-binding
- Data Binding
  - https://blog.thoughtram.io/angular/2016/10/13/two-way-data-binding-in-angular-2.html
- LifeCycle Hooks
  - http://myrighttocode.org/blog/typescript/angular2/angular-2-lifecycle

Key Contacts :

Persistent University :

- Archana Ghatigar
  archana_ghatigar@persistent.co.in

Persistent

Persistent

Thank you!

Persistent Interactive | Persistent University